CHAPTER 8

# Back-end Development

*Connecting Vue · Axios · Forms · JSON Server CRUD*

**Web Application Development**

UTM · FC

2026

CHAPTER OVERVIEW

# Learning Outcomes

1 **Explain Back-end**

Describe the client-server model and the role of a back-end in a modern web application.

2 **Connect Vue to API**

Integrate a Vue 3 application with an HTTP back-end using the Axios client library.

3 **Handle Forms**

Capture, validate and submit user input with v-model and native form events in Vue 3.

4 **Perform CRUD**

Use JSON Server to simulate a REST API and implement Create, Read, Update and Delete operations.

# What is Back-end Development?

*The back-end is everything the user does not see, but that makes the application actually work.*

**Data Storage**

Databases hold persistent information like students, users, and orders.

**Business Logic**

Rules that decide what a request is allowed to do and how to process it.

**Authentication**

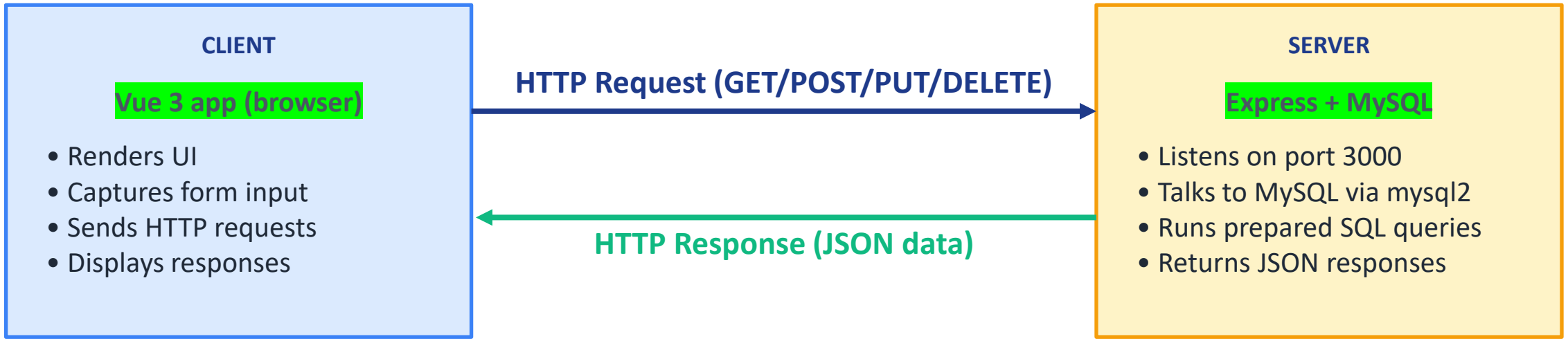
Verifying users, issuing tokens, and protecting private data.

**API Layer**

HTTP endpoints that the front-end calls to read or modify data.

*In this chapter, we simulate a real back-end with a tiny tool called JSON Server, so we can focus on how the front-end talks to it.*

# The Client–Server Model




*The browser never talks to MySQL directly. The Vue front-end sends HTTP to the Express API, which is the only layer that runs SQL against the database.*

COMPARISON

# Front-end vs Back-end

| Aspect | Front-end | Back-end |
|---|---|---|
| Runs in | Browser (HTML/CSS/JS) | Server (Node, Python, etc.) |
| Primary role | User interface & interactions | Data, rules, authentication |
| Example tools | Vue 3, React, Tailwind | Express, Django, JSON Server |
| Visible to user | Yes — what users see & click | No — works behind the scenes |
| In this course | Vue 3 + Axios | JSON Server (simulated REST API) |

*In production, the back-end is typically built with Express, Django, Laravel, or similar. JSON Server lets us focus on the HTTP conversation without writing server-side code.*

CONCEPTS

# What is a REST API?

## REST

Representational State Transfer — a style for designing networked APIs around resources and HTTP verbs.

1 **Resource-based URLs**

Nouns, not verbs. Example: /students, /students/3.

2 **HTTP verbs have meaning**

GET reads, POST creates, PUT/PATCH updates, DELETE removes.

3 **Stateless**

Each request carries everything needed; the server stores no session between calls.

4 **JSON payloads**

Data is exchanged as JSON, the universal format for modern web APIs.

# HTTP Methods You Will Use

| | | |
|---|---|---|
| GET | **Read data**<br>`GET /students` | SAFE · IDEMPOTENT |
| POST | **Create new resource**<br>`POST /students` | NOT IDEMPOTENT |
| PUT | **Replace resource**<br>`PUT /students/3` | IDEMPOTENT |
| PATCH | **Update some fields**<br>`PATCH /students/3` | NOT IDEMPOTENT |
| DELETE | **Remove resource**<br>`DELETE /students/3` | IDEMPOTENT |

***Idempotent*** *= calling it multiple times gives the same result. POST is not idempotent because each call creates a new record.*

# HTTP Status Codes

| 2xx | 3xx | 4xx | 5xx |
|---|---|---|---|
| Success | Redirect | Client Error | Server Error |
| •200 OK<br>•201 Created<br>•204 No Content | •301 Moved Permanently<br>•304 Not Modified | •400 Bad Request<br>•401 Unauthorized<br>•404 Not Found | •500 Internal Server Error<br>•503 Service Unavailable |

*Always inspect the response status in your code — especially in error handlers. Axios treats anything 4xx/5xx as a rejected promise by default.*

PART 01

# Connecting Vue to Backend

*Getting your Vue app to talk to a real API.*

# Fetch vs Axios

| Fetch API | Axios |
| --- | --- |
| ✓ Built into the browser | ✓ Auto JSON parsing |
| ✓ No install needed | ✓ Rejects on 4xx / 5xx |
| ✗ Does not parse JSON automatically | ✓ Interceptors & instances |
| ✗ Does not reject on HTTP errors | ✓ Request / response transforms |
| ✗ No global config / interceptors | ✓ Timeout & cancellation built-in |

**Our choice: Axios**

Axios gives us a clean, config-driven client that behaves predictably and plays nicely with Vue's reactivity. We'll use it throughout this chapter.

# A Quick Look at fetch()

For comparison, here is what a simple GET looks like with the built-in fetch API:

fetch-example.js

```
// Fetch: two awaits, manual JSON parsing, manual error checks
async function getStudents() {
  const response = await fetch('http://localhost:3000/students')
  if (!response.ok) {
    throw new Error('HTTP ' + response.status)
  }
  const data = await response.json()
  return data
}
```

## Why we move on from fetch()

Notice we had to: (1) call response.json() ourselves, (2) manually check response.ok, and (3) write our own error handling.

Axios does all of this out of the box.

FEATURE HIGHLIGHTS

# Why Axios?

1 **Promise-based**
Works seamlessly with async/await.

2 **JSON by default**
Auto-parses JSON and auto-sets Content-Type.

3 **Interceptors**
Run code before every request / after every response.

4 **Instances**
Create configured clients with baseURL, timeout, headers.

5 **Cancellation**
Abort in-flight requests using AbortController.

6 **Universal**
Same API in the browser and in Node.js.

# Installing Axios

In the root of your Vue project, run:

terminal
```
npm install axios
```

Then import Axios in any module that needs to make HTTP calls:

studentApi.js
```
// src/api/studentApi.js
import axios from 'axios'

const response = await axios.get('http://localhost:3000/students')
console.log(response.data)
```

*We will organize all HTTP calls into a dedicated service module — more on that in a moment.*

PART 02

# Axios HTTP Client

*Mastering the HTTP client for Vue.*

# Basic GET Request

basic-get.js

```
import axios from 'axios'

async function loadStudents() {
  try {
    const response = await axios.get('http://localhost:3000/students')
    console.log(response.data)    // the parsed JSON body
    console.log(response.status)  // HTTP status code
  } catch (error) {
    console.error('Failed to fetch:', error.message)
  }
}
```

**response.data**

The parsed JSON body. You rarely need anything else.

**try / catch**

Axios rejects its promise on any 4xx or 5xx response — wrap calls in try / catch or .catch().

# Creating an Axios Instance

An instance avoids repeating baseURL, timeouts, and headers in every call.

studentApi.js

```
// src/api/studentApi.js
import axios from 'axios'

const apiClient = axios.create({
  baseURL: 'http://localhost:3000',
  timeout: 5000,
  headers: {
    'Content-Type': 'application/json',
    Accept: 'application/json'
  }
})

export default apiClient
```

**Best practice**

Keep every back-end call inside one module (for example, src/api/). Components should never import axios directly.

# POST · PUT · PATCH · DELETE

crud.js

```
// CREATE — send a new object, server assigns an id
await apiClient.post('/students', { name: 'Aina', gpa: 3.8 })

// UPDATE (replace all fields)
await apiClient.put('/students/3', {
  name: 'Aina Zahra', matricNo: 'A22CS0033', gpa: 3.9
})

// PATCH (update only the listed fields)
await apiClient.patch('/students/3', { gpa: 3.95 })

// DELETE (remove by id)
await apiClient.delete('/students/3')
```

# Understanding the Response

response.js

```
const res = await apiClient.get('/students/1')

res.data          // parsed JSON body
res.status        // e.g. 200
res.statusText    // e.g. 'OK'
res.headers       // response headers
res.config        // the request config
res.request       // low-level request object
```

**data**

What you almost always want — the

**status**

HTTP status code (200, 404, 500…).

**headers**

Response headers — useful for auth

**config**

Handy for logging the original request.

# Handling Errors

error-handling.js

```
try {
  const res = await apiClient.post('/students', payload)
  // success path
} catch (err) {
  if (err.response) {
    // server responded with a non-2xx status
    console.error('Status:', err.response.status, err.response.data)
  } else if (err.request) {
    // request was sent but no response received
    console.error('No response — is the server running?')
  } else {
    // something else went wrong setting up the request
    console.error('Error:', err.message)
  }
}
```

*Distinguish the three shapes of Axios errors — they all need different user-facing messages.*

# Interceptors

Interceptors run before every request or after every response. Use them for logging, auth, or showing a loader.

interceptors.js

```
// Request interceptor
apiClient.interceptors.request.use((config) => {
  console.log(`[API] ${config.method.toUpperCase()} ${config.url}`)
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// Response interceptor
apiClient.interceptors.response.use(
  (res)  => res,
  (err) => {
    if (err.response?.status === 401) location.href = '/login'
    return Promise.reject(err)
  }
)
```

# Loading Data in a Vue Component

StudentList.vue

```
<script setup>
import { ref, onMounted } from 'vue'
import { getStudents } from '../api/studentApi.js'

const students = ref([])
const loading  = ref(false)
const error    = ref('')

async function load() {
  loading.value = true
  try {
    const res = await getStudents()
    students.value = res.data
  } catch (e) { error.value = 'Could not load.' }
  finally { loading.value = false }
}

onMounted(load)
</script>
```

THINGS TO WATCH FOR

# Common Axios Pitfalls

### CORS errors

If the server is on a different origin, enable CORS or use a proxy. JSON Server enables it by default.

### Losing this

Axios uses arrow functions — no this binding issues, but watch for it in class components.

### Forgetting .data

response.data is what you want. res itself is the whole envelope.

### Hard-coded URLs

Always go through a service module with a shared baseURL for portability.

### Swallowed errors

Never write catch(() => {}). Log, report, or surface to the user.

### Race conditions

If a user triggers many requests quickly, cancel stale ones with AbortController.

PART 03

# Form Handling in Vue

*Capturing and validating user input in Vue.*

# v-model — Two-way Data Binding

v-model keeps a reactive variable and an input element in sync — automatically.

Greeting.vue

```
<template>
  <input v-model="name" />
  <p>Hello, {{ name }}!</p>
</template>

<script setup>
import { ref } from 'vue'
const name = ref('')
</script>
```

## Under the hood

v-model is short for :value="name" and @input="name = $event.target.value".

## Works with any input

Text, textarea, checkbox, radio, select — Vue picks the right event and property automatically.

# v-model Modifiers

| Modifier | Effect | Example |
|---|---|---|
| .lazy | Sync on change instead of input. | `<input v-model.lazy="msg" />` |
| .number | Cast the value to a number. | `<input v-model.number="age" />` |
| .trim | Auto-trim whitespace. | `<input v-model.trim="username" />` |

modifiers.html

```
<input v-model.number="form.gpa"  type="number" step="0.01" />
<input v-model.trim="form.name"   type="text" />
<input v-model.lazy="form.notes"  type="text" />
```

# Working with Different Inputs

inputs.vue

```
<!-- SELECT -->
<select v-model="form.faculty">
  <option value="">-- Select --</option>
  <option value="FSKSM">FSKSM (Computing)</option>
  <option value="FKE">FKE (Electrical)</option>
</select>

<!-- CHECKBOX (boolean) -->
<input type="checkbox" v-model="form.active" />

<!-- CHECKBOX GROUP (array) -->
<input type="checkbox" value="vue"   v-model="skills" />
<input type="checkbox" value="axios" v-model="skills" />

<!-- RADIO -->
<input type="radio" value="male"   v-model="form.gender" />
<input type="radio" value="female" v-model="form.gender" />
```

# Submitting the Form

SubmitExample.vue

```
<template>
  <form @submit.prevent="onSubmit">
    <input v-model.trim="form.name" placeholder="Name" />
    <input v-model.number="form.gpa" type="number" step="0.01" />
    <button type="submit">Save</button>
  </form>
</template>

<script setup>
import { ref } from 'vue'
const form = ref({ name: '', gpa: 0 })
function onSubmit() {
  // send form.value to the API
}
</script>
```

**.prevent**

Stops the browser's default form submission (which would reload the page).

STRATEGIES

# Client-side Validation

### 1 HTML5 attributes

required, min, max, type="email", pattern — free and automatic.

### 2 Custom JS function

Write a validate() function that sets an errors object. Flexible and testable.

### 3 Computed properties

For cross-field rules (e.g. passwordConfirm must match password).

### 4 Schema libraries

Use VeeValidate, Vuelidate or Zod on larger forms.

*Always validate on the server too — the browser is not a security boundary.*

# Validation in Action

StudentForm.vue

```
const errors = ref({})
const matricRegex = /^[A-Z][0-9]{2}[A-Z]{2}[0-9]{4}$/

function validate() {
  const e = {}
  if (!form.value.name.trim())
    e.name = 'Name is required.'
  if (!matricRegex.test(form.value.matricNo))
    e.matricNo = 'Format: A21CS0001'
  if (form.value.gpa < 0 || form.value.gpa > 4)
    e.gpa = 'GPA must be 0.00 – 4.00.'
  errors.value = e
  return Object.keys(e).length === 0
}

async function onSubmit() {
  if (!validate()) return
  await createStudent(form.value)
}
```

PART 04

# Express + MySQL CRUD

*A real REST API backed by a persistent database.*

# Why Express + MySQL?

*Pair Vue with a real Node.js API and a persistent MySQL database — the same stack you would deploy in production.*

1 **Persistent data**

Records survive restarts — stored on disk in MySQL InnoDB tables.

2 **Industry stack**

Node.js + Express + MySQL is one of the most widely used web stacks.

3 **Auto IDs**

MySQL AUTO_INCREMENT assigns ids on every INSERT, just like JSON

4 **SQL power**

Filter, sort and aggregate with full SQL — joins, indexes, aggregates.

5 **CORS middleware**

The express + cors package lets the Vite dev server call the API.

6 **Bundled with Laragon**

MySQL ships with Laragon; only Node.js and npm packages are extra.

TWO COMMANDS

# Set up the Backend

Install backend dependencies (one time):

```
terminal

  cd server && npm install express mysql2 cors
```

Start the API server (every time you code):

```
terminal
 node server/index.js      (or use: npm run server)
```

**What happens?**

Express listens on port 3000, accepts JSON requests from the Vue app, runs prepared SQL through mysql2, and sends JSON responses back.

**Tip**

Make sure Laragon's MySQL service is green BEFORE starting the API, otherwise the first request will fail with ECONNREFUSED 127.0.0.1:3306.

# Schema and seed data

sql/schema.sql

```
CREATE DATABASE IF NOT EXISTS student_records;
USE student_records;

CREATE TABLE students (
  id       INT AUTO_INCREMENT PRIMARY KEY,
  matricNo VARCHAR(10)  NOT NULL UNIQUE,
  name     VARCHAR(120) NOT NULL,
  course   VARCHAR(150) NOT NULL,
  faculty  VARCHAR(30)  NOT NULL,
  gpa      DECIMAL(3,2) NOT NULL,
  email    VARCHAR(120) NOT NULL,
  year     TINYINT      NOT NULL,
  active   TINYINT(1)   NOT NULL DEFAULT 1
) ENGINE=InnoDB;

INSERT INTO students (matricNo, name, course, faculty,
                      gpa, email, year, active) VALUES
  ('A21CS0001', 'Ahmad Zulkarnain bin Hassan',
   'BSc Computer Science', 'FSKSM', 3.75,
   'ahmad.zulkarnain@graduate.utm.my', 3, 1);
```

*Run this script once in HeidiSQL (Laragon). The Vue front-end never sees SQL — all queries go through the Express API.*

# READ — Fetching Data

| URL | Meaning |
|---|---|
| `GET /students` | All students |
| `GET /students/3` | Student with id 3 |
| `GET /students?faculty=FSKSM` | Filter by exact match |
| `GET /students?_sort=gpa&_order=desc` | Sorting |
| `GET /students?q=aisyah` | Full-text search |

studentApi.js

```
export const getStudents   = ()   => apiClient.get('/students')
export const getStudent    = (id) => apiClient.get(`/students/${id}`)
export const searchStudents = (q) => apiClient.get(`/students?q=${q}`)
```

POST

# CREATE — Adding Data

create.js

```
// Send the new student WITHOUT an id — the server assigns one.
const payload = {
  matricNo: 'A24CS0100',
  name: 'Nurul Huda',
  course: 'Computer Science', faculty: 'FSKSM',
  gpa: 3.8, email: 'nurul@graduate.utm.my', year: 1, active: true
}
const res = await apiClient.post('/students', payload)
```

## Request

POST http://localhost:3000/students with the new student as a JSON body.

## Response

201 Created plus the full record including the auto-assigned id from MySQL (returned as result.insertId).

## Push into local state

After a successful POST, append res.data to your reactive students array so the UI updates immediately.

# UPDATE — Changing Data

update.js

```
// PUT — send the ENTIRE record. Missing fields are wiped.
await apiClient.put('/students/3', {
  matricNo: 'A22EE0015', name: 'Lim Wei Jian',
  course: 'Electrical Engineering', faculty: 'FKE',
  gpa: 3.65, email: 'lim@graduate.utm.my', year: 2, active: true
})

// PATCH — send only the fields that change.
await apiClient.patch('/students/3', { gpa: 3.65 })
```

### Use PUT when

You have the complete object in memory (typical after editing a form). Safer because the UI always reflects the full state.

### Use PATCH when

Only one or two fields change (e.g. toggling active), or the record is large and you want smaller payloads.

*Match the id in the URL — /students/3 — with the id in your state. Never invent ids.*

# DELETE — Removing Data

delete.js

```
async function handleDelete(id) {
  if (!confirm('Delete this student?')) return
  await apiClient.delete(`/students/${id}`)
  students.value = students.value.filter(s => s.id !== id)
}
```

### Confirm first

Always ask for confirmation before a destructive action — a misclick should not wipe user data.

### Update local state

After a successful DELETE, remove the item from your reactive array so the list updates without an extra round-trip.

***Advanced tip**: for a better UX, do an optimistic update — remove from the UI first, rollback if the request fails.*

THE END-TO-END FLOW

# Putting It All Together

1 **User types**

Form inputs are bound to reactive state with v-model.

2 **Submit**

@submit.prevent triggers a validate() and then a call to the API service.

3 **Axios**

The configured instance sends the HTTP request to the Express API on port 3000.

4 **Server**

Express runs a prepared SQL query through mysql2; MySQL persists the change and returns rows.

5 **Response**

Axios resolves; you update the reactive students array.

6 **UI updates**

Vue reactively re-renders the list — no manual DOM code.

# Summary — Key Takeaways

- A back-end exposes data through HTTP endpoints. Vue talks to it with an HTTP client.
- Axios is the preferred client: auto JSON, interceptors, instances, clean error handling.
- Organize every HTTP call in a dedicated api/ module; components should never import axios directly.
- v-model binds a reactive variable to any form input. Combine with .trim, .number, .lazy modifiers.
- @submit.prevent + a validate() function is enough for most forms; reach for libraries on large ones.
- JSON Server turns a .json file into a REST API, letting you practise full CRUD without writing a server.

**Next up**

Lab hands-on — build the Student Records Manager step by step.

NEXT SESSION

# Lab Hands-on Preview

### 1 Scaffold

Create a Vue 3 project with Vite and install Axios and JSON Server.

### 2 Mock API

Write db.json and start JSON Server on port 3000.

### 3 API Service

Implement src/api/studentApi.js with an Axios instance and CRUD functions.

### 4 Form

Build StudentForm.vue with v-model, validation, and a submit handler.

### 5 List

Build StudentList.vue — show, edit and delete records.

### 6 Wire Up

Compose everything in App.vue; test the full CRUD flow.

*References · Vue 3 Docs (vuejs.org) · Axios (axios-http.com) · JSON Server (github.com/typicode/json-server)*